Local-first system blueprint | Proposed v0.2 target

Atlas - Architecture Document

Version: 0.2.0 (TARGET ARCHITECTURE, NOT CURRENT IMPLEMENTATION)

Date: 2026-10-14 | Repository: Swayamjain1240/Atlas | Status: Design approved; migration pending

1. Executive summary

Atlas is a privacy-first Personal Life Operating System. It indexes user-approved files and browser history into a private memory layer, supports semantic search and source-grounded chat, and eventually accommodates additional connectors. This document supersedes the v0.1 architecture technology choices while preserving its product goals.

The proposed application comprises one local product distributed as coordinated processes: React UI, Node.js/Express API, and an internal Python AI worker, with SQLite and ChromaDB storing data on the device. Strictly speaking this is a local-first modular multi-process monolith, not a single-process monolith.

2. Locked technology decisions

- Frontend: React 18, Vite, Tailwind CSS, Three.js and GSAP. Retain the existing React views and improve them incrementally.
- Main backend: Node.js 20+ and Express for HTTP API, validation, authorization, application policies and SQLite persistence.
- AI runtime: Python 3.11+, an internal FastAPI adapter, Sentence-Transformers all-MiniLM-L6-v2 (384 dimensions), RAG orchestration and Ollama with llama3.2:3b.
- Storage: ChromaDB for vectors, SQLite for metadata, chat sessions, configuration and ingestion jobs. Persist both locally.
- Local-only by default: no remote inference, analytics, or cloud database. First installation/model downloads may require connectivity.
- Packaging: one local product; the Express gateway owns user-facing routes; the Python worker is a private loopback-only dependency.

3. Boundaries and ownership

- frontend/: presentation, accessible navigation, 3D graph, animation, chat, dashboard, settings; never direct access to files, ChromaDB or Python.
- backend/: Express route/controller/service modules, auth, permission checks, input validation, consent management, SQLite migrations, jobs metadata and response contracts.
- aiServices/: Python embedding, ingestion extraction/chunking, vector indexing/search, RAG assembly, Ollama calls and ChromaDB lifecycle.
- tests/: frontend, backend, aiServices, integration, E2E, security and performance verification.
- docs/: architecture, requirements, system design and master control file. scripts/: setup, dev, audit and packaging helpers.
- data/: local SQLite, ChromaDB and caches. Ignore data and secrets in git; prefer per-user data directory outside the clone.

4. Repository target tree

- backend/src/{config,routes,controllers,middleware,services,database,models,validators,utils}/; app.js; server.js.
- frontend/src/{assets,components,pages,layouts,hooks,context,services,animations,three,styles,utils}/; App.jsx; main.jsx.
- aiServices/app/{api,config,ingestion,embeddings,rag,llm,vectorstore,services,utils}/; main.py.
- tests/{backend,frontend,aiServices,integration,e2e,security,performance}/.
- docs/{atlas_architecture,atlas_system_design,atlas_requirements,atlas_master}.{md,pdf}.
- scripts/, data/ (gitignored), README.md, .gitignore, .env.example and LICENSE.
- These are TARGET directories; absence from the current repository is expected until implementation.

5. Component interactions

- Browser -> Express via same-origin /api/v1 endpoints. API authenticates, validates and authorizes every non-public request.
- Express -> internal Python FastAPI via 127.0.0.1 only, authenticated by a separate internal secret. Never expose Python to the LAN.
- Node -> SQLite as the sole application-layer writer and transaction owner.
- Python -> ChromaDB as the sole vector-store writer. Vector indexes have document IDs that map back to SQLite memory records.
- Python -> local Ollama API on loopback. No user content is sent to hosted LLMs.
- Backend returns standardized response objects including sources, timings and safe errors; frontend renders plain text by default.

6. Primary workflows

6.1 Ingestion

- User explicitly selects an allowed directory or supported browser profile; default is no automatic scanning.
- Express records permission and a bounded ingestion job, then forwards a validated, constrained request to Python.
- Python discovers eligible content, reads it with file-type parsers, extracts text, chunks with overlap, embeds and upserts vectors.
- Express upserts memory metadata/job state into SQLite through a defined coordination contract; no silent cross-database transaction claims.
- Track source fingerprint and content hash. Re-ingest changed files idempotently, delete stale chunks, preserve source provenance.

6.2 Query

- User query -> authenticated Express endpoint -> Python query embed -> Chroma similarity search -> context assembly -> local Ollama response.
- Return answer, ranked citations, relevance values, model ID, processing time and explicit empty-context behavior.
- Express persists chat turn/feedback in SQLite if user enabled conversation history; no unsupported claim of persistent chat in the current repo.

6.3 Deletion and retention

- User-initiated delete removes vector chunks, SQLite references and cached derived content; keep failures visible for retry.
- Source revocation stops watchers and optional future ingestion; define whether existing indexed memories are retained or deleted.

7. API security and privacy boundary

- Node binds 127.0.0.1 by default; no implicit LAN access. Strict loopback origin allowlist, authenticated protected routes, conservative CORS.
- Short-lived or session-scoped local API credentials are preferred over persistent secrets in localStorage.
- Python worker checks internal credential plus peer binding. Do not rely on CORS as authentication or assume localhost is trusted.
- Limit directories with canonical real paths, no symlink escapes, explicit extensions, maximum bytes, depth, concurrency and file-count quotas.
- Redact secrets, prompts and full documents from logs. No telemetry or external fonts/CDNs in offline production packaging.
- Maintain dependency scanning, pre-commit checks, safe error formats, local data permissions, backup/restore and secure defaults.

8. UI and visualization boundary

- One-viewport authenticated page shell; internal bounded panel scrolling/pagination is allowed where necessary for accessible content.
- Tailwind dark deep-space palette with purple/indigo. GSAP handles lifecycle-safe motion; Three.js renders the memory graph.
- 3D nodes must correspond to real indexed memories when data exists. Current random decorative particles are not a functional graph.
- Provide reduced-motion preference, fallback when WebGL is unavailable, accessible keyboard controls and responsive screens.

9. Deployment and development

- Development: Vite dev server, Express gateway and Python worker started with documented local scripts; Ollama runs separately.
- Production target: built React assets served locally, Express gateway, private Python worker, local Ollama, user-data folder.
- Supported target OS: Windows 10+, macOS 12+ and Ubuntu 20.04+, validated by OS-specific smoke tests.
- Docker Compose remains optional and does not override local-only/privacy rules.

10. Migration strategy and compatibility

- M1 snapshot main and validate existing UI / FastAPI health endpoint. Preserve existing PDFs as git history.
- M2 introduce Express app and standardized contracts without deleting the Python backend.
- M3 move existing Python security/business HTTP concerns to Express and retain Python AI-specific modules in aiServices/.
- M4 wire one vertical slice: sample text -> embed -> Chroma -> retrieve -> Ollama -> React chat.

- M5 add ingestion sources, SQLite persistence, deletion, timeline, real 3D graph and settings.
- M6 remove old FastAPI public routes only after contract, security and E2E tests pass. Do not move/delete data automatically.
- Each milestone requires Duck review against master, requirements and test evidence.

11. Scope guardrails

- v0.2 core: text/PDF/DOCX/code ingestion, browser history metadata, local embeddings/vector search, grounded chat, citations and basic memory management.
- Deferred: email/Slack/Discord/WhatsApp connectors, autonomous agents, cloud sync, distributed mode, MCP, biometric auth and Tauri desktop packaging.
- Performance targets are aspirations requiring hardware-specific benchmarks; do not state that a CPU-only Llama 3.2 system meets 2s p95 without measurements.